

Blind Recovery for Objective-Aware Scheduling

A bridge note: what GO-EC-3's 94.6% means if you don't care about water-filling

Observation Theory program — systems bridge note

August 2026 — companion to ledger row G0-EC-3 (G0-P-2026-087, sealed, ALL PASS 5/5).

The problem, in systems terms

You run a state estimator — a Kalman filter, say — over a pool of candidate sensors, and each step you can only afford to sample a few of them. Which few? The textbook answers maximize information: pick the sensors that shrink the estimate covariance the most (*logdet*), or that cut its trace (*iso-trace*). Both are **consumer-blind**: they optimize the estimate in the abstract, not the thing that actually *reads* the estimate downstream — your controller, your detector, your serving policy, whatever consumes the state and has an objective of its own. When the downstream objective is anisotropic (it cares about some directions of the state far more than others), consumer-blind scheduling spends your sensor budget in the wrong place.

The catch: you usually can't *open* the downstream consumer. It may be a learned model, a third-party component, or a policy too complex to differentiate by hand. So the question is: **can you schedule sensors for a downstream objective you can only probe as a black box — and is it worth the probing cost?**

The idea (no water-filling required)

Treat the consumer as a black box you can *query*: feed it a state estimate, read back its loss. With a handful of finite-difference queries you recover a local model of what it cares about — a positive semidefinite *read operator* $\hat{P}_C$, the curvature of its loss in state space. Then you score any candidate sensor set by how much it reduces the error the consumer will feel, which is the single scalar

$$\text{tr}(\hat{P}_C \Sigma),$$

Σ being the Kalman covariance you already track. Greedily pick the sensors that drive this down. That's the whole pipeline: probe, compose, schedule. No knowledge of the consumer's internals; no analytic objective; and every probe query is **charged against the same sensor budget**, so the comparison is like-for-like.

What GO-EC-3 measured

A single sealed, pre-registered governed run (fresh seed 20260819, 20 fresh systems, probe cost charged), two consumer arms:

- **Positive control (analytic consumer).** Here the optimal schedule is *known* in closed form, so the blind pipeline has a target to match, not beat. It captured **94.6%** of the known optimum’s gain (gate $\geq 75\%$). In plain terms: probing recovered almost all of what you would get if you had the consumer’s source code.
- **Black-box consumers (no analytic optimum).** A frozen low-rank MLP and a smoothed-threshold consumer, where no closed form exists. The blind schedule beat the best consumer-*agnostic* policy by **16.3%** at matched budget (gate $\geq 8\%$).
- **Ordering.** Across 61 system pairs, the blind score predicted which schedule wins **86.9%** of the time (gate $\geq 65\%$).
- **No free lunch.** Probing a consumer with *shuffled* inputs gave only +0.08 (a near-null control); an *anti* policy that schedules against the recovered operator was worst per arm; the probe charge itself cost +0.0030 (a sensitivity row, no verdict weight).

gate	quantity	result	bar
P1	match of known optimum (positive control)	0.946	≥ 0.75
P2	improvement over best agnostic (black box)	0.163	≥ 0.08
P3	trace-matched ordering, 61 pairs	0.869	≥ 0.65
	shuffled-consumer / anti controls	clean	—
	probe charge (sensitivity)	+0.0030	—

Why a systems person should care

The marketable claim is not about geometry. It is: **you can make a scarce sampling budget serve a downstream objective you cannot open, automatically, by querying it — and recover almost all of the gain you would get from white-box access, while paying for the probes.** The same recipe — probe the consumer, compose with what you already track, schedule — is objective-agnostic in the useful sense: swap the Kalman covariance for a cache’s access statistics and you have objective-aware eviction; swap it for a feature store’s uncertainty and you have query scheduling. It turns “we don’t know what the downstream wants” from a blocker into a bounded probing cost.

What it is not (scope, stated plainly)

The systems in GO-EC-3 are **synthetic**: linear-time-invariant dynamics with planted consumers. The greedy scheduler carries **no optimality claim** (submodular greedy is a heuristic here). It is a **single** governed run, reported per its pre-registration. The result earns the *mechanism* — blind recovery works, and it pays for itself — not a deployment number. The physical-endpoint version, on a real estimator and a real downstream, is a separate campaign (Campaign 5, its own registry ID) and has not been run. Read this note as “here is why the mechanism is worth a real trial,” not “here is a benchmark.”